



BTPR1103 Java Programming II

Group Project (30%)

Year 2026 Semester B

Project Instructions

1. This Project will contribute 30% to your final grade
2. Groups must consist of 3-4 members.
3. All members must be of same subject code and studies.
4. Project is due on week 12 **14th August 2026**.

CLO	Description
CLO2	Demonstrate the concepts and techniques in Java event-driven programming, use the exception handling and database in Java application.
CLO3	Adopt the knowledge of Java GUI programming to develop a functional GUI program.

Submission Instruction

- Each group must submit:
 1. **Source code** of the complete system,
 2. **Project report** in PDF format,
 3. **Project distribution** of each member,
 4. **Screenshots** of the developed system.
- The report should be submitted as **Portable Document Format (PDF)**.
- Groups are encouraged to upload their source code to **GitHub** and include the repository link in the report.
- Name your file as "<ProjectCode>_2026B_project_<studentID>_<studentID>". For example: *BTPR1103_2026B_project_B123456B_ B123457B_ B123456A.pdf*.
- If any assignment is submitted after the deadline, 50% deducted from the total assignment marks. Marks will also be deducted for not adhering to the instructions.

Problem Statement

Community-related issues often require simple digital systems to improve service coordination, resource management, and user access to information. Examples include managing donations, arranging volunteer participation, coordinating appointments, or organizing local support activities. A **functional Java GUI application** can help solve such problems by providing a clear user interface for data entry, interaction, and record viewing.

This project therefore requires students to design and develop a **GUI-based Java application** that addresses a **community-impact problem** through a practical and user-friendly system. The system must demonstrate the use of **GUI programming**, **event-driven programming**, and **exception handling**, which are all part of the Java II course content.

Project Requirements

Students are required to plan and develop **one integrated Java application** using **Java Swing or JavaFX**, with appropriate GUI components, event handling, and exception handling. The project should focus on a **working prototype** rather than a complex commercial-scale solution.

Each group member must contribute **at least one major feature or module** to the system. All features must be integrated into **one complete working application**. During the presentation, each member must be able to explain and demonstrate their contribution clearly.

The project must emphasize the following:

- development of a **functional Java GUI application**,
- implementation of **event handling** for user interaction,
- use of **exception handling and input validation**,
- modular feature development and system integration by all group members.

Groups may use file handling, data storage or other simpler storage methods if appropriate.

Suggested Project Scope

Each group is advised to develop a system with:

- one main dashboard or menu

- three to five core features,
- multiple GUI screens or windows
- input forms and output areas such as text areas, labels, lists, or tables
- event handling for major user actions
- exception handling for invalid or incomplete input.

Sample Project Ideas

Students may propose their own topic, provided that it has **meaningful benefit to the community** and is suitable for implementation as a Java GUI application. Similar topics or ideas are allowed. The following are examples of acceptable project topics:

1. Community Food Donation and Request System

A system that allows users to register, submit food donations, request food assistance, and view donation/request records. An administrator screen may be included to review and update request status.

2. Blood Donation Appointment System

A system that allows users to register as donors, book appointments, view appointment history, and check simple eligibility information.

3. Volunteer Activity Coordination System

A system that allows volunteers to register, sign up for community activities, view participation history, and allow organizers to manage volunteer records.

System Requirements

1. GUI Application

Develop the system using **Java Swing or JavaFX**. The application must include appropriate GUI components such as:

- labels, text fields, text areas,
- buttons, radio buttons, check boxes, combo boxes,
- lists, tables, or other suitable output components,
- multiple screens/windows where appropriate.

The GUI should be user-friendly, organized, and clearly support the intended tasks of the system.

2. Event Handling and Validation

The system must implement event handling for user actions such as:

- add / register,
- search / view,
- update / delete,
- submit / clear / navigate.

Groups must also validate user input and apply exception handling where appropriate, for example:

- empty field validation,
- invalid numeric input,
- duplicate record detection (if applicable),
- invalid logical input or incomplete form submission.

Clear feedback messages should be provided to guide the user.

3. Functional Requirements

The project must contain **three to five core features** relevant to the selected topic. Typical functions may include:

- registration of users or records,
- searching and displaying information,
- updating or managing records,
- generating a simple summary or history display,
- file save/load or basic record storage.

All features must work correctly and be integrated into **one complete application**.

Group Contribution Requirement

Each student must contribute **at least one major feature or module**. Examples of major features include:

- login or registration module,
- booking/request/donation form,
- search or display module,
- report/history screen,
- dashboard/navigation and system integration.

Task Requirements

Each group is required to:

1. Choose a suitable community-impact project topic.
2. Design the system structure and screen flow.
3. Develop a functional Java GUI application using Java Swing or JavaFX.
4. Implement event handling and exception handling in the application.
5. Integrate all members' modules into one complete system.
6. Prepare a short report explaining the developed system.
7. Present and demonstrate the final project as a group.

Deliverables

Each group must submit the following:

1. Source Code

Complete Java source code or link to source code of the final project.

2. Project Report

A short report in **PDF format** that includes:

- background of the selected problem,
- objectives of the system,

- explanation of system features,
- explanation of GUI design,
- event handling used,
- exception handling and input validation,
- screenshots of the system,
- challenges faced and possible improvements,
- contribution of each group member.

3. Presentation Slides

Slides used during the project presentation.

4. Demonstration

A live demonstration of the running system during presentation.

Presentation Requirements

Each group is required to give a **10–15 minute presentation**. The presentation should include:

- introduction to the selected community problem,
- overview of the developed system,
- explanation of key features,
- demonstration of the GUI application,
- explanation of event handling and exception handling,
- explanation of each member's contribution.

All members must participate in the presentation and demonstration.

Recommended Report Structure

Groups are encouraged to organize the report as follows:

1. Introduction / Problem Background

2. Objectives of the System
3. System Features
4. GUI Design Explanation
5. Event Handling Used
6. Exception Handling and Validation
7. Screenshots of the System
8. Testing / Challenges Faced
9. Individual Contribution
10. Conclusion

Marking Rubric

Criteria	CLO	Excellent	Good	Fair	Poor
GUI Design and User Interface (20%)	CL03	17–20: Interface is highly organized, user-friendly, visually clear, and uses appropriate GUI components, layouts, and navigation effectively.	13–16: Interface is functional and mostly well organized, with minor layout or usability issues.	8–12: Interface is basic but usable. Some components are poorly arranged or unclear.	0–7: Interface is incomplete, poorly designed, or difficult to use.
Functionality and Feature Implementation (20%)	CL03	17–20: All required features are fully implemented and working correctly. The system effectively addresses the selected problem.	13–16: Most required features are implemented and working with minor issues.	8–12: Some features are implemented, but there are missing or incomplete functions.	0–7: Few features work correctly, or the system is largely incomplete.
Event Handling (15%)	CL02	13–15: Event handling is implemented correctly and consistently for all major actions.	9–12: Most event handling works correctly, with minor weaknesses.	5–8: Basic event handling is present but incomplete or inconsistent.	0–4: Event handling is incorrect, weak, or non-functional.
Exception Handling and Validation (10%)	CL02	9–10: Validation is thorough and exception handling is correctly implemented with clear messages.	7–8: Validation and exception handling are present with minor weaknesses.	4–6: Some validation is included, but there are gaps or unclear error handling.	0–3: Little or no validation or exception handling is demonstrated.

Code Quality and Integration (10%)	CL03	9–10: Code is well-structured, readable, modular, and all features are properly integrated.	7–8: Code is mostly organized and integrated, with minor issues.	4–6: Code works in parts but lacks consistency or smooth integration.	0–3: Code is disorganized or features are not integrated into one complete system.
Report (10%)	CL03	9–10: Report is clear, well-organized, and complete with good explanation, screenshots, and contribution details.	7–8: Report is organized and mostly complete, with minor omissions.	4–6: Report is basic and partially explains the project.	0–3: Report is poorly written, incomplete, or missing.
Presentation and Demonstration (10%)	CL03	9–10: Presentation is confident, clear, professional, and the demonstration runs smoothly. All members participate.	7–8: Presentation is understandable and the demonstration works with minor issues.	4–6: Presentation is basic or unclear. Demonstration is limited or uneven.	0–3: Presentation is poorly prepared or incomplete. Demonstration fails or is missing.
Individual Contribution (5%)	CL03	5: Student shows strong ownership of assigned feature and clear understanding of the project.	4: Student contributes well and shows acceptable understanding.	2–3: Student shows limited contribution or partial understanding.	0–1: Student contributes very little or cannot explain their work.